



**Department of Electronic &
Telecommunication Engineering**

University of Moratuwa

Sri Lanka

EN3150 Assignment 03
**Simple convolutional neural
network to perform classification**

Group Members:

210707L	Wickramasinghe M M M
210687X	Weerasinghe C N
210321X	Kumarasinghe R D
210521K	Ranasinghe R P R U

12/12/2024

Contents

1	CNN for Image Classification	2
2	Comparison with State-of-the-Art Networks	9
2.1	Resnet-18	9
2.2	VGG-16	11
2.3	trade-offs, advantages, and limitations of using a custom model versus a pre-trained model	13
2.3.1	Custom Models	13
2.3.2	Comparison Table	14

1 CNN for Image Classification

Question 2

Selected dataset: **RealWaste**

**RealWaste**
Donated on 10/17/2023

An image classification dataset of waste items across 9 major material types, collected within an authentic landfill environment.

Dataset Characteristics	Subject Area	Associated Tasks
Image	Computer Science	Classification
Feature Type	# Instances	# Features
-	4752	-



Question 3

Splitting the dataset into training, validation, and testing subsets using a ratio of 60% for training and 20% each for validation and testing sets.

```
dataset = RealWasteDataset("realwaste-main/RealWaste/")

train_size, validation_size = int(0.6 * len(dataset)), int(0.2 * len(dataset)) # 80% training
test_size = len(dataset) - train_size - validation_size
train_dataset, validation_dataset, test_dataset = random_split(dataset, [train_size, validation_size, test_size])
print("Train:", len(train_dataset))
print("Validation:", len(validation_dataset))
print("Test:", len(test_dataset))

# Create DataLoaders
BATCH_SIZE = 32
train_dataloader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
validation_dataloader = DataLoader(validation_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_dataloader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

Train: 2851
Validation: 950
Test: 951
```

Question 4

```
class ConvNeuralNet(nn.Module):
    def __init__(self, num_classes):
        super(ConvNeuralNet, self).__init__()
        self.conv_layer1 = nn.Conv2d(in_channels=3, out_channels=8, kernel_size=5)
        self.max_pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv_layer2 = nn.Conv2d(in_channels=8, out_channels=1, kernel_size=5)
        self.max_pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(625, 128) # Updated flattened size
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(0.2)
        self.fc2 = nn.Linear(128, 9)

    def forward(self, x):
        out = self.conv_layer1(x)
        out = self.relu(out)
        out = self.max_pool1(out)

        out = self.conv_layer2(out)
        out = self.relu(out)
        out = self.max_pool2(out)

        out = out.reshape(out.size(0), -1)

        out = self.fc1(out)
        out = self.relu1(out)
        out = self.dropout1(out)
        out = self.fc2(out)
        return out

# Example of initializing the network
model = ConvNeuralNet(9).to('cuda' if torch.cuda.is_available() else 'cpu')
```

Question 5

```
ConvNeuralNet(
  (conv_layer1): Conv2d(3, 8, kernel_size=(5, 5), stride=(1, 1))
  (max_pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv_layer2): Conv2d(8, 1, kernel_size=(5, 5), stride=(1, 1))
  (max_pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=625, out_features=128, bias=True)
  (relu1): ReLU()
  (dropout1): Dropout(p=0.2, inplace=False)
  (fc2): Linear(in_features=128, out_features=9, bias=True)
)
```

Question 6

- **ReLU activation function** :is widely used in convolutional layers to introduce non-linearity while maintaining low computational complexity, resulting in faster performance. Its efficiency stems from the simplicity of its threshold operation. Unlike sigmoid or tanh functions, ReLU helps mitigate the vanishing gradient problem, ensuring effective gradient propagation during backpropagation.
- **The Softmax activation function**: is typically employed in the output layer to convert logits into probabilities. However, it is not directly applied in the final layer during loss computation. Instead, it works seamlessly with the commonly used cross-entropy loss function.

Parameter	Layer/Type	Value
Activation Function	Convolutional Layers	ReLU
	Fully Connected Layers	ReLU
	Output Layer	Softmax
Kernel Size	First Convolutional Layer	3×3
	Second Convolutional Layer	3×3
	Third Convolutional Layer	3×3
Filter Size	First Convolutional Layer	32 filters
	Second Convolutional Layer	64 filters
	Third Convolutional Layer	128 filters
Fully Connected Layer	First FC Layer: Input Size	$128 \times 28 \times 28$
	First FC Layer: Output Size	256
	Second FC Layer: Input Size	256
	Second FC Layer: Output Size	9
Dropout Rate		No Dropout Layer
Output Layer Size		9 (Number of Classes)

Table 1: Network Configuration Summary

Question 7

Training the model using the training data for 20 epochs and plot training and validation loss for with respect to epoch.

Optimizer used: Adam optimizer

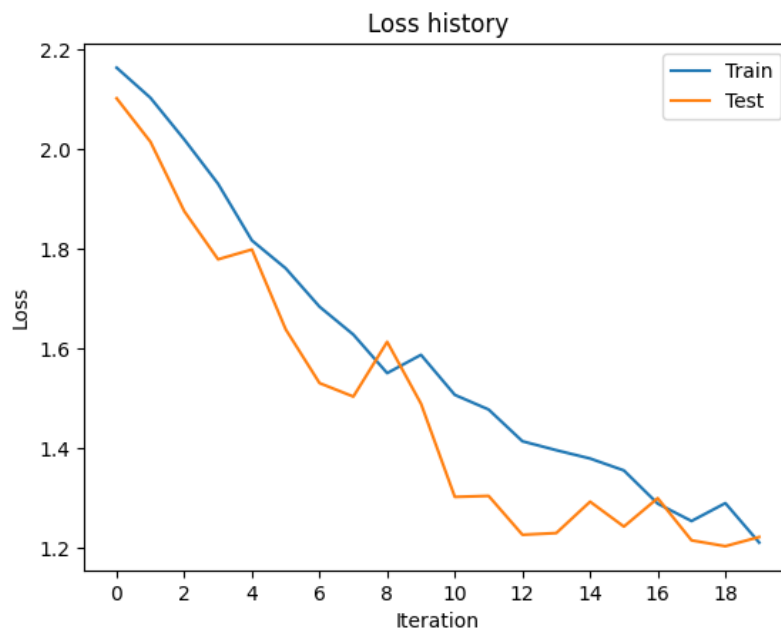


Figure 1: Training and Validation Loss

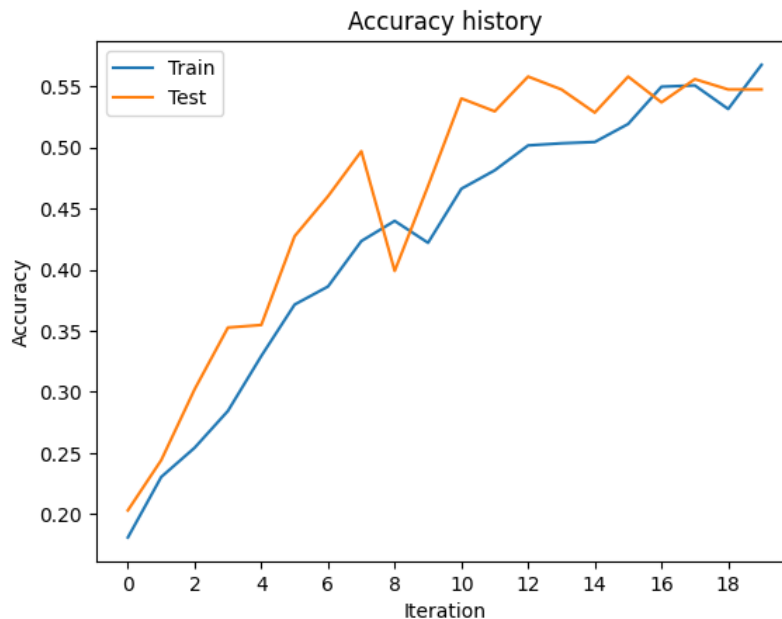


Figure 2: Training and Validation Accuracy

Question 8

The Adam optimizer leverages adaptive learning rates, which adjust dynamically based on historical gradient information. This allows it to achieve faster convergence and produce better solutions, especially in scenarios where the magnitudes of gradients vary significantly. Unlike Stochastic Gradient Descent (SGD), where a fixed learning rate must be predefined and remains constant throughout training, Adam adapts the learning rate for each parameter individually.

A key feature of Adam is gradient scaling, which helps stabilize the training process by accounting for the magnitude of past gradients. This scaling reduces the likelihood of oscillations or divergence during backpropagation, making the optimizer more robust to changes in gradient magnitude. As a result, Adam significantly minimizes the need for manual tuning of the learning rate, a common challenge when using SGD. This makes it particularly suitable for complex models and datasets where determining an optimal fixed learning rate is difficult.

Question 9

Sparse Categorical Cross Entropy is a commonly used loss function in multi-class classification problems. This uses integer encoding which is more efficient when we have a lot of categories or labels which would consume a huge amount of resources(RAM).

Question 10

Train Accuracy: 63.98%

Test Accuracy: 54.78%

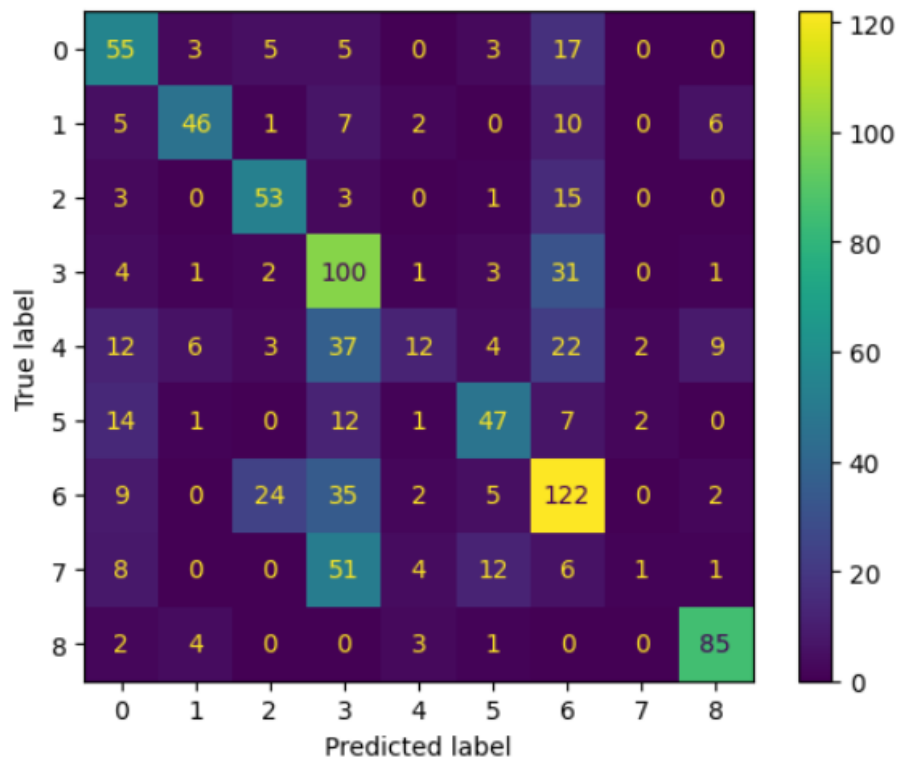


Figure 3: Confusion Matrix

Question 11

Small Learning Rate (0.0001)

A very small learning rate results in slower training as the updates to the model parameters are minimal in each step. Consequently, the loss decreases gradually, which makes the training process more time-consuming and can delay convergence to the optimal solution.

Large Learning Rate (0.1)

When the learning rate is excessively large, the training and validation losses may oscillate or even diverge. This occurs because the parameter updates overshoot the optimal point in the loss function, preventing the model from stabilizing and hindering convergence to a minimum.

Moderate Learning Rate (0.001)

A moderate learning rate offers a balanced approach, allowing the model to update parameters efficiently while avoiding overshooting the optimal point. This enables faster and more stable convergence, making it an ideal choice for reliable model training.

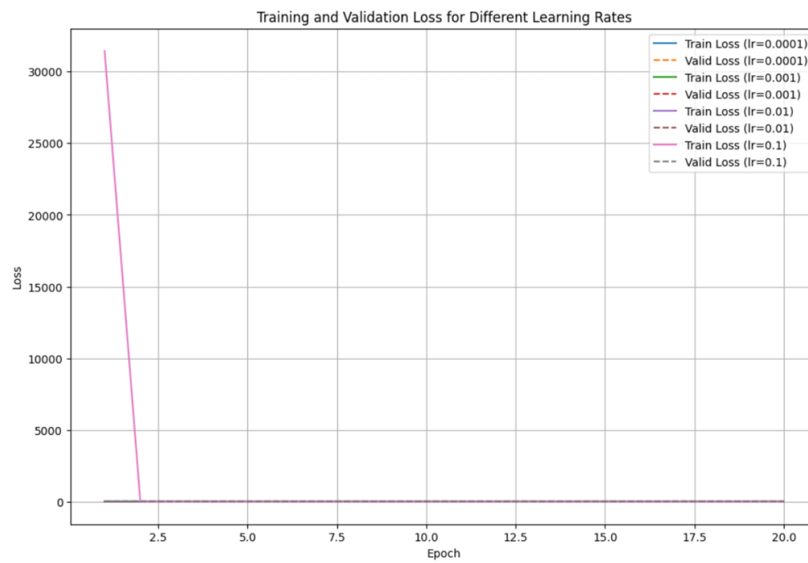


Figure 4: Training and Validation Loss for Different Learning Rates

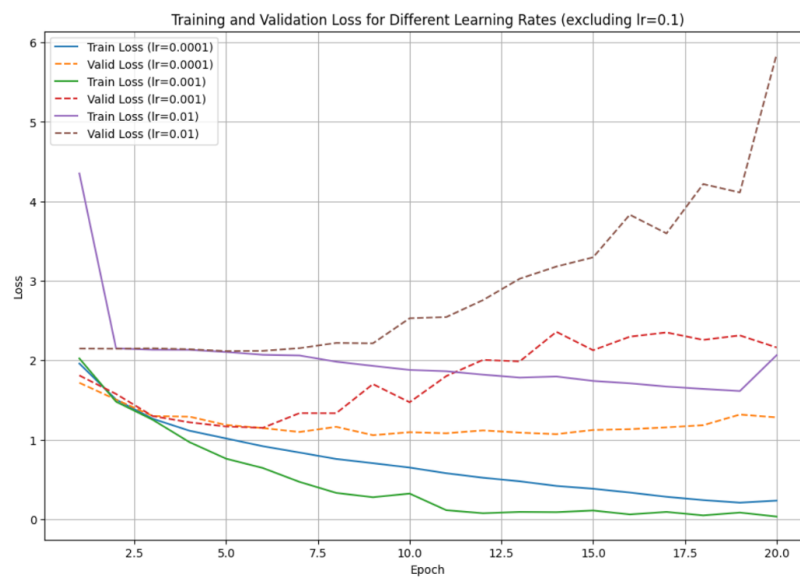


Figure 5: Training and Validation Loss for Different Learning Rates(

2 Comparison with State-of-the-Art Networks

Question 12

We chose Resnet-18 and VGG 16

2.1 Resnet-18

Train Accuracy: 52.43%

Test Accuracy: 33.18%

```
Epoch 1 / 20, Train Loss: 1.2572589072916243, Test Loss: 1.2665621519088746, Train Accuracy: 0.5580498070852332, Test Accuracy: 0.5726315789473684
Epoch 2 / 20, Train Loss: 0.9704642706447177, Test Loss: 1.5256344477335613, Train Accuracy: 0.6478428621536303, Test Accuracy: 0.5178947368421053
Epoch 3 / 20, Train Loss: 1.0215985119342803, Test Loss: 2.1790023922920225, Train Accuracy: 0.6601192564012627, Test Accuracy: 0.48526315789473684
Epoch 4 / 20, Train Loss: 0.7443489999172059, Test Loss: 1.2106615881125131, Train Accuracy: 0.7471062784987723, Test Accuracy: 0.6
Epoch 5 / 20, Train Loss: 0.6428454435533948, Test Loss: 1.3087791085243226, Train Accuracy: 0.7804279200280604, Test Accuracy: 0.6189473684210526
Epoch 6 / 20, Train Loss: 0.5262662344508701, Test Loss: 1.2448945800463358, Train Accuracy: 0.8235706769554543, Test Accuracy: 0.6389473684210526
Epoch 7 / 20, Train Loss: 0.5615802865889338, Test Loss: 1.3070878724257151, Train Accuracy: 0.8067344791301297, Test Accuracy: 0.5736842105263158
Epoch 8 / 20, Train Loss: 0.46184536814689636, Test Loss: 1.2657751282056173, Train Accuracy: 0.8533847772711329, Test Accuracy: 0.6347368421052632
Epoch 9 / 20, Train Loss: 0.3784584299557739, Test Loss: 1.598607563972473, Train Accuracy: 0.8793405822518414, Test Accuracy: 0.5652631578947368
Epoch 10 / 20, Train Loss: 0.4126882057223055, Test Loss: 1.9371302763621012, Train Accuracy: 0.8635566467905997, Test Accuracy: 0.4905263157894737
Epoch 11 / 20, Train Loss: 0.3617785542375512, Test Loss: 1.2880491693814595, Train Accuracy: 0.8775868116450368, Test Accuracy: 0.6273684210526316
Epoch 12 / 20, Train Loss: 0.31723545835249956, Test Loss: 1.7328214287757873, Train Accuracy: 0.8926692388635566, Test Accuracy: 0.588421052631579
Epoch 13 / 20, Train Loss: 0.25467810299661425, Test Loss: 1.3027528166770934, Train Accuracy: 0.9172220273588214, Test Accuracy: 0.6568421052631579
Epoch 14 / 20, Train Loss: 0.20477365089787378, Test Loss: 1.7541490336259207, Train Accuracy: 0.9470361276745002, Test Accuracy: 0.5357894736842105
Epoch 15 / 20, Train Loss: 0.27595090725355675, Test Loss: 1.6627281069755555, Train Accuracy: 0.9116099614170466, Test Accuracy: 0.5273684210526316
Epoch 16 / 20, Train Loss: 0.3085833449330595, Test Loss: 1.4800669391949972, Train Accuracy: 0.8982813048053314, Test Accuracy: 0.6021052631578947
Epoch 17 / 20, Train Loss: 0.13968317488001453, Test Loss: 1.3947633365790049, Train Accuracy: 0.9600140301648544, Test Accuracy: 0.6157894736842106
Epoch 18 / 20, Train Loss: 0.18770923593805897, Test Loss: 1.2543093075354894, Train Accuracy: 0.9449316029463346, Test Accuracy: 0.6663157894736842
Epoch 19 / 20, Train Loss: 0.2244484943234258, Test Loss: 1.7130130350589752, Train Accuracy: 0.9277446509996492, Test Accuracy: 0.5631578947368421
Epoch 20 / 20, Train Loss: 0.1269243414203326, Test Loss: 3.351924022038778, Train Accuracy: 0.9579095054366888, Test Accuracy: 0.36
```

Figure 6: Training and validation loss values for each epoch



Figure 7: Training and Validation Loss

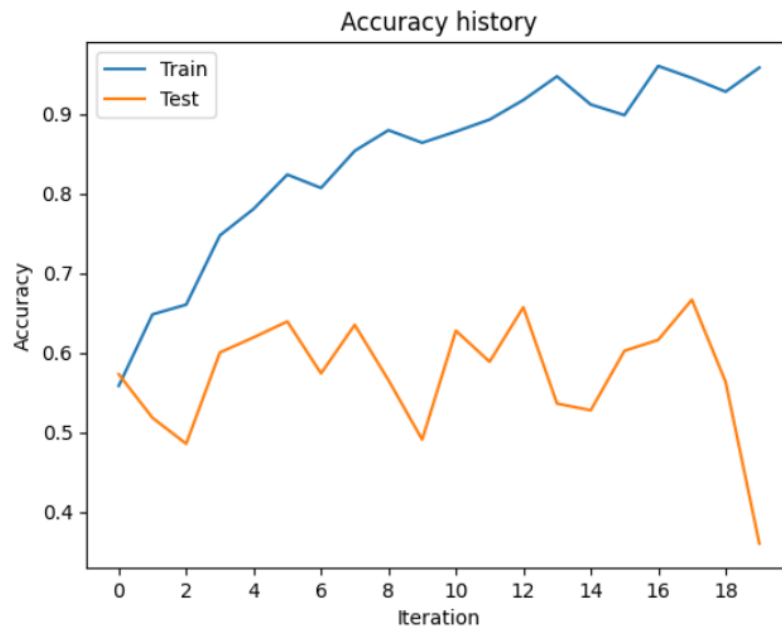


Figure 8: Training and Validation Accuracy

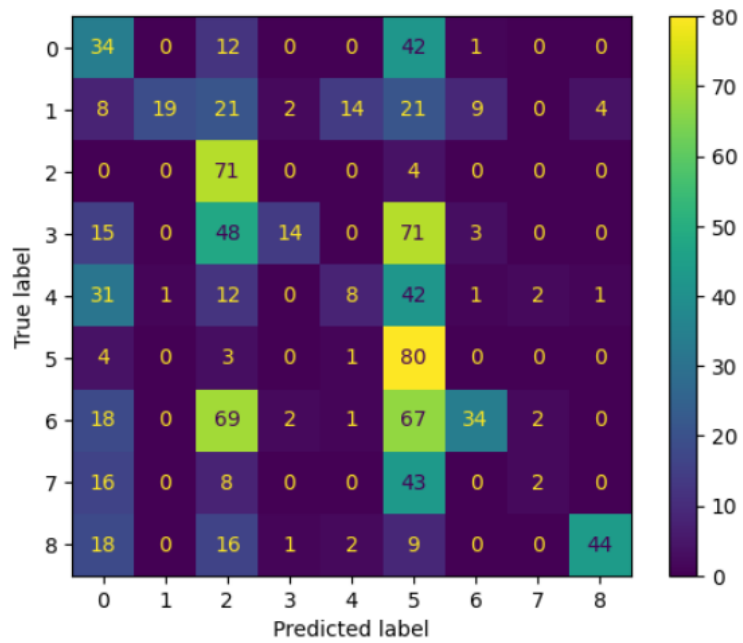


Figure 9: Confusion Matrix

2.2 VGG-16

Train Accuracy: 66.73%

Test Accuracy: 56.32%

```
Epoch 1 / 20, Train Loss: 2.248660537931654, Test Loss: 2.062850844860077, Train Accuracy: 0.17222027358821465, Test Accuracy: 0.22631578947368422
Epoch 2 / 20, Train Loss: 1.9470009830262927, Test Loss: 1.8513290166854859, Train Accuracy: 0.26797614871974745, Test Accuracy: 0.25684210526315787
Epoch 3 / 20, Train Loss: 1.8087235755390592, Test Loss: 1.787570635477702, Train Accuracy: 0.31673097158891617, Test Accuracy: 0.3273684210526316
Epoch 4 / 20, Train Loss: 1.6324538853433397, Test Loss: 1.7193208813667298, Train Accuracy: 0.3952998947737636, Test Accuracy: 0.3778947368421053
Epoch 5 / 20, Train Loss: 1.550548379951053, Test Loss: 1.5139518618583678, Train Accuracy: 0.4279200280603297, Test Accuracy: 0.44842105263157894
Epoch 6 / 20, Train Loss: 1.5394657909870149, Test Loss: 1.6208757599194845, Train Accuracy: 0.4373903893370747, Test Accuracy: 0.4073684210526316
Epoch 7 / 20, Train Loss: 1.384140564997991, Test Loss: 1.6008416374524435, Train Accuracy: 0.4766748509294984, Test Accuracy: 0.4031578947368421
Epoch 8 / 20, Train Loss: 1.3335209449132284, Test Loss: 1.4468699892361958, Train Accuracy: 0.49736934408979305, Test Accuracy: 0.4884210526315789
Epoch 9 / 20, Train Loss: 1.32954125073221, Test Loss: 1.5534913023312886, Train Accuracy: 0.4952648193616275, Test Accuracy: 0.4694736842105263
Epoch 10 / 20, Train Loss: 1.213282616270913, Test Loss: 1.255340540409088, Train Accuracy: 0.5443703963521571, Test Accuracy: 0.5326315789473685
Epoch 11 / 20, Train Loss: 1.1411142309506734, Test Loss: 1.4015991727511088, Train Accuracy: 0.5713784636969484, Test Accuracy: 0.4905263157894737
Epoch 12 / 20, Train Loss: 1.110529703564114, Test Loss: 1.344589763879776, Train Accuracy: 0.5843563661873027, Test Accuracy: 0.5231578947368422
Epoch 13 / 20, Train Loss: 1.0979314141803318, Test Loss: 1.2202432751655579, Train Accuracy: 0.5882146615222729, Test Accuracy: 0.5547368421052632
Epoch 14 / 20, Train Loss: 1.1280703836017185, Test Loss: 1.2633466482162476, Train Accuracy: 0.5917222027358822, Test Accuracy: 0.5515789473684211
Epoch 15 / 20, Train Loss: 1.0864738298787011, Test Loss: 1.2383884926637014, Train Accuracy: 0.5885654156436338, Test Accuracy: 0.5505263157894736
Epoch 16 / 20, Train Loss: 1.0284144865141975, Test Loss: 1.2270003656546276, Train Accuracy: 0.6222378112942827, Test Accuracy: 0.531578947368421
Epoch 17 / 20, Train Loss: 1.0017661540044678, Test Loss: 1.0970067838827768, Train Accuracy: 0.6197825324447562, Test Accuracy: 0.5978947368421053
Epoch 18 / 20, Train Loss: 0.9538518912262387, Test Loss: 1.0832837124665577, Train Accuracy: 0.648895124517713, Test Accuracy: 0.6094736842105263
Epoch 19 / 20, Train Loss: 0.9511624468697442, Test Loss: 1.2321808020273843, Train Accuracy: 0.648895124517713, Test Accuracy: 0.5684210526315789
Epoch 20 / 20, Train Loss: 0.9250887460178799, Test Loss: 1.3145018676916758, Train Accuracy: 0.6534549280954052, Test Accuracy: 0.5526315789473685
```

Figure 10: Training and validation loss values for each epoch

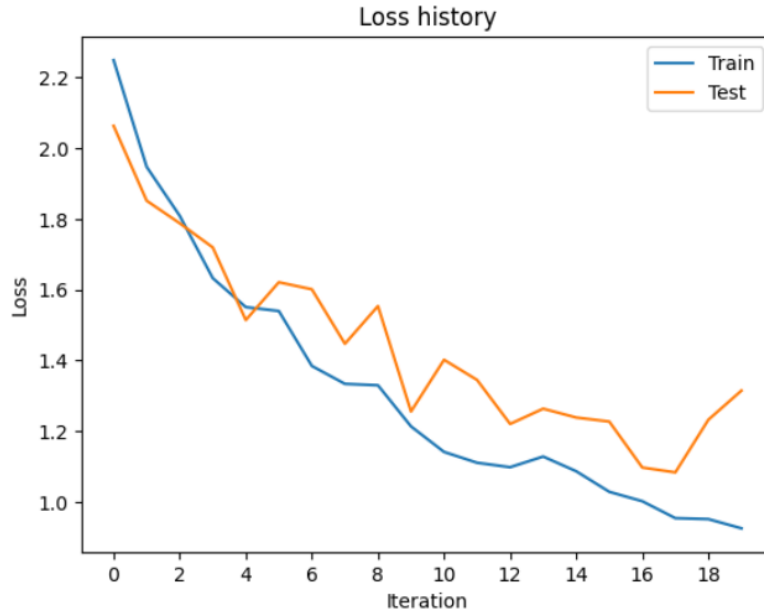


Figure 11: Training and Validation Loss

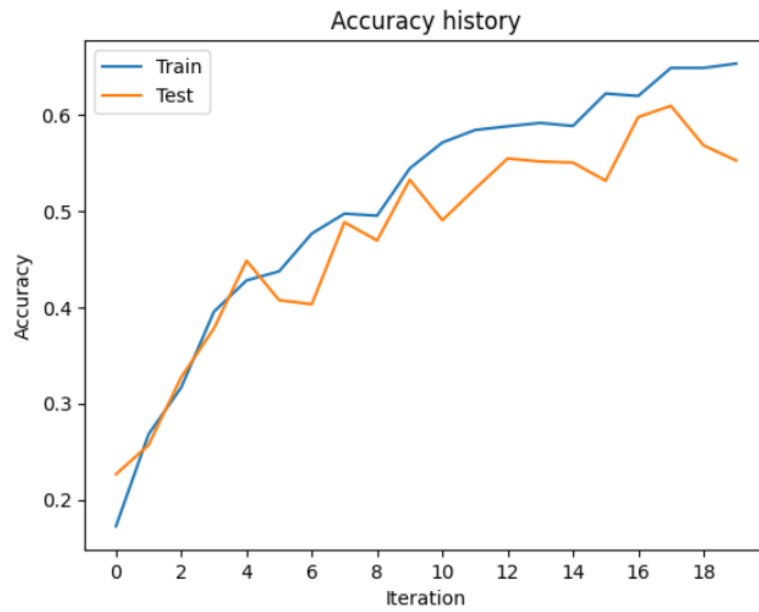


Figure 12: Training and Validation Accuracy

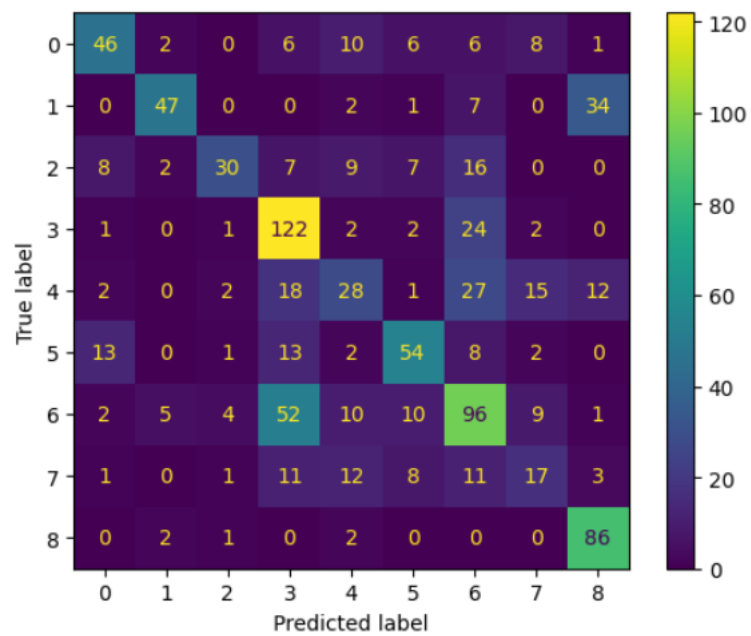


Figure 13: Confusion Matrix

Question 18

Test accuracy is highest in VGG-16, But lower in Resnet-18 than custom model

2.3 trade-offs, advantages, and limitations of using a custom model versus a pre-trained model

In deep learning, selecting between a custom model and a pre-trained model depends on several factors such as the available dataset, computational resources, and the specific task. Both approaches have their own advantages and limitations.

2.3.1 Custom Models

A custom model is designed and trained from scratch for a specific task. This allows for greater flexibility but comes with certain challenges.

Advantages of Custom Models

- **Tailored for the Task:** Custom models can be specifically designed to meet the requirements of the given problem, allowing for greater adaptability.
- **No Dependency on Large Datasets:** Custom models are suitable for tasks where you have a smaller, domain-specific dataset that does not align with the datasets used to train pre-trained models.
- **Full Control Over Architecture:** Since the model is built from the ground up, you have complete control over the architecture and can experiment with different designs.

Limitations of Custom Models

- **Data Requirements:** Custom models typically require large amounts of data to achieve good performance. A lack of sufficient data may lead to poor generalization and overfitting.
- **High Training Cost:** Training custom models from scratch is computationally expensive and time-consuming, often requiring powerful hardware and extended training periods.
- **Risk of Poor Generalization:** If the data is limited or not diverse enough, the model may struggle to generalize to new, unseen data.

Pre-trained Models

Pre-trained models are trained on large datasets and can be fine-tuned for a specific task, saving time and resources.

Advantages of Pre-trained Models

- **Reduced Training Time:** Pre-trained models require much less time to train. Fine-tuning only involves adjusting the final layers to adapt the model to the new task.
- **Better Performance with Less Data:** Since pre-trained models have already learned useful features from large datasets, they can perform well even with smaller datasets.
- **Lower Resource Requirements:** Fine-tuning a pre-trained model requires less computational power compared to training a model from scratch.
- **Improved Generalization:** Pre-trained models typically generalize better as they are trained on large, diverse datasets.

Limitations of Pre-trained Models

- **Domain Mismatch:** If the pre-trained model was trained on data that is very different from your specific domain (e.g., medical images), performance may be suboptimal.
- **Limited Flexibility:** The architecture of a pre-trained model is fixed, and you may not be able to customize it entirely for very specific tasks.
- **Overfitting with Small Data:** When fine-tuning a pre-trained model on a very small dataset, there is a risk of overfitting, especially if the model is large.

2.3.2 Comparison Table

Criteria	Custom Model	Pre-trained Model
Data Requirements	Requires large, task-specific data	Can work well with limited data, requires fine-tuning
Training Time	High, as it is trained from scratch	Low, as it is fine-tuned
Computational Resources	High, needs powerful hardware	Lower, only fine-tuning is required
Performance	Depends on data size and model design	High, especially for tasks similar to original training
Flexibility	High, can be customized fully	Lower, limited by original model architecture
Generalization Ability	May struggle with limited data	High, due to pre-training on large datasets
Domain-Specific Adaptation	Fully adaptable to specific domains	May require fine-tuning for domain mismatch

Table 2: Comparison between Custom Models and Pre-trained Models

References

- [1] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems*, vol. 25, 2012. DOI: <https://doi.org/10.1145/3065386>.
- [2] I. S. Single, Sam, and R. Raad, "RealWaste," UCI Machine Learning Repository, 2023. DOI: <https://doi.org/10.24432/C5SS4G>.

Note that: All the codes are uploaded to the [github repository](#).